

Oui, Karl. Il faut les séparer dès le départ, sinon on finit avec une table `posts` ingérable où commerce, storytelling, modération et paiements deviennent trop couplés.

Je partirais sur ces domaines métier distincts :

- **identity** : `profiles`, `seller_profiles`, `addresses`, `verification_status`
- **commerce** : `listings`, `listing_images`, `listing_categories`, `listing_attributes`, `offers`
- **storytelling** : `stories`, `story_modes`, `relationship_contexts`, `story_reactions`, `comments`
- **next chapter** : `next_chapters`, `chapter_goals`, `chapter_contributions`, `chapter_updates`
- **transactions** : `orders`, `order_items`, `payments`, `payouts`, `refunds`, `shipping_events`
- **social** : `follows`, `saved_listings`, `notifications`
- **messaging** : `conversations`, `conversation_members`, `messages`
- **trust & safety** : `reports`, `moderation_cases`, `moderation_events`, `blocked_users`
- **platform/admin** : `audit_logs`, `feature_flags`, `system_events`

Le point essentiel est que `listing` **et** `story` **ne soient pas la même chose**.

Une annonce pourrait exister sans histoire publique :

```
listing
├─ seller_id
├─ title
├─ description
├─ category_id
├─ price
├─ condition
├─ status
└─ ...
```

Puis séparément :

```
story
├─ listing_id
├─ author_id
├─ story_mode
├─ relationship_context_id
├─ text
└─ anonymous
```

|— moderation_status

Ça permet plus tard de masquer une histoire problématique **sans retirer l'objet de la vente**, ou inversement de retirer une annonce commerciale tout en conservant éventuellement une version éditoriale autorisée de l'histoire.

Même séparation pour `Next Chapter`.

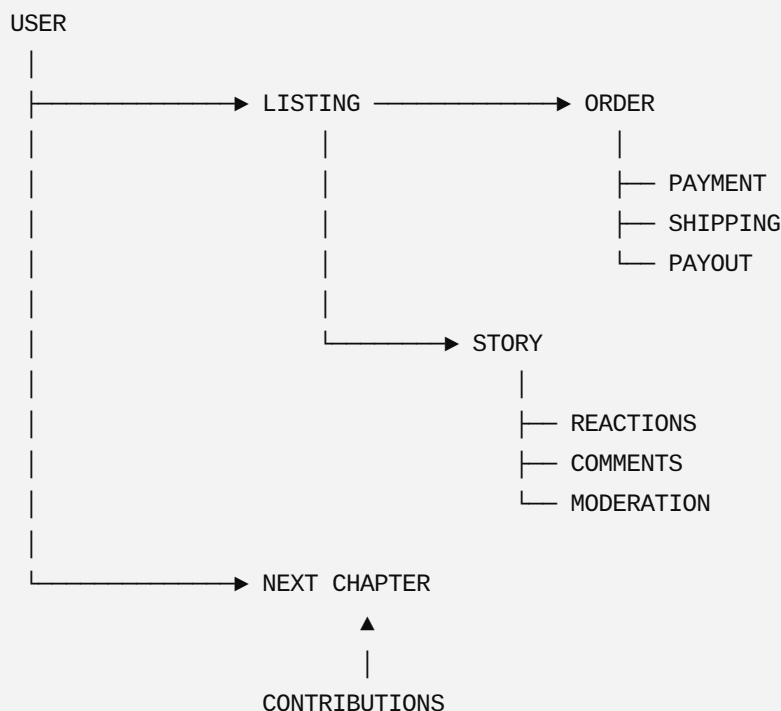
```
next_chapter
|— owner_id
|— title
|— description
|— target_amount
|— visibility
|— status
└ ...
```

et :

```
chapter_contribution
|— chapter_id
|— order_item_id
|— amount
└ created_at
```

Donc une vente réelle alimente automatiquement le projet futur. On évite de simplement stocker un champ `fund_progress = 690`, qui deviendrait incohérent dès qu'un remboursement intervient.

L'architecture logique devient alors :



▲
|
SOLD ITEMS

Et je séparerais même `relationship_context` **de** `story`.

Exemple :

```
relationship_context
├ id
├ slug
├ label
├ emoji
└ is_sensitive
```

Valeurs possibles :

```
cheated_on
ghosted
wedding_cancelled
divorce
moving_out
peaceful_goodbye
terrible_gift
at_least_they_had_taste
what_was_i_thinking
other
```

Ça évite d'inscrire "Cheated on me" en texte libre dans 40 000 lignes et ça permet ensuite analytics, traduction, recommandations et modération.

Même logique pour les modes :

```
story_mode
clean_break
little_tea
full_story
```

Et **anonymous** ne devrait probablement pas être un `story_mode`.

Techniquement, je le traiterais comme un paramètre de visibilité :

```
identity_visibility:
public
limited
anonymous
```

Parce qu'une histoire peut très bien être :

Full Story + Anonymous

ou :

Little Tea + Public

C'est une petite décision de modélisation, mais elle évite une incohérence importante plus tard.

Pour le MVP Supabase, je commencerais donc avec environ **20-25 tables bien définies**, plutôt que 6 énormes tables génériques. Et je conserverais PostgreSQL comme source de vérité : les montants, états de commande, progression des fonds et droits d'accès ne doivent jamais dépendre uniquement du frontend.

La prochaine étape logique est maintenant de définir **le schéma PostgreSQL/Supabase complet avec relations, clés étrangères, enums, index, RLS et workflow des statuts**. C'est vraiment le blueprint technique à faire avant de commencer Next.js.